

1) Read a dataset from the user and i. Use the Find-S algorithm to find the most specific hypothesis that is consistent with the positive examples. Ii. What is the final hypothesis after processing all the positive examples? Using the same dataset, apply the Candidate Elimination algorithm. Determine the final version space after processing all examples (both positive and negative). What are the most specific and most general hypotheses in the version space?

Program 1: Find-S Algorithm

```
import pandas as pd

# Read the dataset
data = pd.read_csv("EnjoySport.csv")

# Get attributes and target
concepts = data.iloc[:, :-1].values
target = data.iloc[:, -1].values

# Initialize hypothesis
hypothesis = None

# Apply Find-S Algorithm
for i in range(len(concepts)):
    if target[i] == "Yes":

        if hypothesis is None:
            hypothesis = list(concepts[i])

        else:
            for j in range(len(hypothesis)):
                if hypothesis[j] != concepts[i][j]:
                    hypothesis[j] = "?"
```

```
# Display the final hypothesis
print("Final Hypothesis:")
print(hypothesis)
```

Output:

Final Hypothesis:
['Sunny', 'Warm', '?', 'Strong', '?', '?']

Program 2: Candidate Elimination Algorithm

```
import pandas as pd
# Read dataset
data = pd.read_csv("EnjoySport.csv")

concepts = data.iloc[:, :-1].values
target = data.iloc[:, -1].values

# Initialize S and G
S = list(concepts[0])
G = [['?' for i in range(len(S))] for j in range(len(S))]

for i in range(len(concepts)):

    if target[i] == "Yes":

        # Generalize S
        for j in range(len(S)):
            if S[j] != concepts[i][j]:
                S[j] = '?'
                G[j][j] = '?'

    else:

        # Specialize G
```

```

    for j in range(len(S)):
        if S[j] != concepts[i][j]:
            G[j][j] = S[j]
        else:
            G[j][j] = '?'

# Remove empty hypotheses
G_final = []
for g in G:
    if g != ['?'] * len(S):
        G_final.append(g)

print("Specific Hypothesis (S):")
print(S)

print("\nGeneral Hypotheses (G):")
for g in G_final:
    print(g)

```

Output:

Specific Hypothesis (S):

['Sunny', 'Warm', '?', 'Strong', '?', '?']

General Hypotheses (G):

['Sunny', '?', '?', '?', '?', '?']

['?', 'Warm', '?', '?', '?', '?']

Dataset:

jupyter enjoysport.csv Last Checkpoint: 11 months ago								
File Edit View Settings Help								
Delimiter: ,								
	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport	
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes	
2	Sunny	Warm	High	Strong	Warm	Same	Yes	
3	Rainy	Cold	High	Strong	Warm	Change	No	
4	Sunny	Warm	High	Strong	Cool	Change	Yes	

2) Read a supervised dataset and use bagging and boosting technique to classify the dataset. Indicate the performance of the model.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# ---- Bagging ----
bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(),
    n_estimators=50,
    random_state=42
)
bagging.fit(X_train, y_train)
y_pred_bagging = bagging.predict(X_test)
bagging_acc = accuracy_score(y_test, y_pred_bagging)

# ---- Boosting (AdaBoost with SAMME to avoid warning) ----
boosting = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=50,
```

```

    algorithm="SAMME", # ✓ avoids FutureWarning
    random_state=42
)
boosting.fit(X_train, y_train)
y_pred_boosting = boosting.predict(X_test)
boosting_acc = accuracy_score(y_test, y_pred_boosting)

# ---- Results ----
print("Bagging Accuracy:", bagging_acc)
print("Boosting Accuracy:", boosting_acc)

print("\n--- Bagging Classification Report ---")
print(classification_report(y_test, y_pred_bagging,
target_names=iris.target_names))

print("\n--- Boosting Classification Report ---")
print(classification_report(y_test, y_pred_boosting,
target_names=iris.target_names))

```

Output:

Bagging Accuracy: 0.9333333333333333				
Boosting Accuracy: 0.9333333333333333				
--- Bagging Classification Report ---				
	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.88	0.93	0.90	15
virginica	0.93	0.87	0.90	15
accuracy			0.93	45
macro avg	0.93	0.93	0.93	45
weighted avg	0.93	0.93	0.93	45
--- Boosting Classification Report ---				
	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.88	0.93	0.90	15
virginica	0.93	0.87	0.90	15
accuracy			0.93	45
macro avg	0.93	0.93	0.93	45
weighted avg	0.93	0.93	0.93	45

3) Read a dataset and use an example-based method (such as RIPPER or CN2) to generate a set of classification rules . Apply the FOIL algorithm (First-Order Inductive Learner) to learn first-order rules for predicting.

Program:

CN2

```
import pandas as pd

# -----
# 1. Create the dataset
# -----

data = [
    ("sparrow", "bird", 1, 1),
    ("eagle", "bird", 1, 1),
    ("penguin", "bird", 0, 0),
    ("bat", "mammal", 1, 1),
    ("cat", "mammal", 0, 0),
    ("butterfly", "insect", 1, 1)
]

columns = ["name", "type", "has_wings", "can_fly"]

df = pd.DataFrame(data, columns=columns)

print("Dataset:")
print(df)

# -----
# 2. Generate CN2-like rules
# -----

print("\nCN2-like Classification Rules:")

for attribute in ["type", "has_wings"]:
```

```

for value in df[attribute].unique():

    # Select examples satisfying the condition
    subset = df[df[attribute] == value]

    # Check whether all selected examples are positive
    if subset["can_fly"].all():

        print(
            f"IF {attribute} = {value} "
            f"THEN can_fly = 1"
        )

```

Output:

Dataset:				
	name	type	has_wings	can_fly
0	sparrow	bird	1	1
1	eagle	bird	1	1
2	penguin	bird	0	0
3	bat	mammal	1	1
4	cat	mammal	0	0
5	butterfly	insect	1	1

CN2-like Classification Rules:				
IF type = insect THEN can_fly = 1				
IF has_wings = 1 THEN can_fly = 1				

FOIL

```

import pandas as pd
import math

# -----
# 1. Create the dataset
# -----

data = [
    ("sparrow", "bird", 1, 1),
    ("eagle", "bird", 1, 1),
    ("penguin", "bird", 0, 0),

```

```

    ("bat", "mammal", 1, 1),
    ("cat", "mammal", 0, 0),
    ("butterfly", "insect", 1, 1)
]
columns = ["name", "type", "has_wings", "can_fly"]
df = pd.DataFrame(data, columns=columns)

print("Dataset:")
print(df)
# -----
# 2. Create first-order facts
# -----
facts = {}
for _, row in df.iterrows():
    name = row["name"]
    facts[name] = set()
    # First-order predicate: type(X, value)
    facts[name].add(
        f"type({name},{row['type']})"
    )
    # First-order predicate: has_wings(X)
    if row["has_wings"] == 1:
        facts[name].add(
            f"has_wings({name})"
        )
# -----
# 3. Separate positive and negative examples
# -----
positive = []
negative = []

for _, row in df.iterrows():

```

```

    if row["can_fly"] == 1:
        positive.append(row["name"])
    else:
        negative.append(row["name"])
print("\nPositive Examples:")
print(positive)
print("\nNegative Examples:")
print(negative)

# -----
# 4. Check whether an example satisfies a literal
# -----

def satisfies(example, literal):

    fact = literal.replace("X", example)
    return fact in facts[example]

# -----
# 5. Find examples covered by a rule body
# -----

def covered(body, examples):
    result = []
    for example in examples:

        if all(
            satisfies(example, literal)
            for literal in body
        ):
            result.append(example)
    return result

# -----
# 6. Calculate FOIL information
# -----

def information_gain(p0, n0, p1, n1, t):

```

```
if p1 == 0:
    return 0

# Information before adding literal
before = math.log2(
    p0 / (p0 + n0)
)
# Information after adding literal
after = math.log2(
    p1 / (p1 + n1)
)
gain = t * (after - before)
return gain

# -----
# 7. Calculate gain for a candidate literal
# -----
def calculate_gain(body, literal,
                   current_positive,
                   current_negative):

    # Number of positive examples before
    p0 = len(current_positive)

    # Number of negative examples before
    n0 = len(current_negative)

    # Add candidate literal
    new_body = body + [literal]

    # Positive examples covered after adding literal
    new_positive = covered(
        new_body,
        current_positive
```

```

)
# Negative examples covered after adding literal
new_negative = covered(
    new_body,
    current_negative
)
p1 = len(new_positive)
n1 = len(new_negative)

# t = number of positive examples
# covered after adding literal
t = p1
gain = information_gain(
    p0, n0, p1, n1, t
)
return gain, new_positive, new_negative
# -----
# 8. FOIL Algorithm
# -----
def foil(positive, negative):
    learned_rules = []
    # Positive examples not yet covered
    uncovered_positive = set(positive)
    while uncovered_positive:
        body = []
        current_positive = set(
            uncovered_positive
        )
        current_negative = set(
            negative
        )
        print("\n-----")
        print("Learning a new rule")

```

```

print("-----")
while current_negative:

    candidates = [
        "has_wings(X)",
        "type(X,bird)",
        "type(X,mammal)",
        "type(X,insect)"
    ]

    best_literal = None
    best_gain = float("-inf")

    print("\nCurrent rule body:", body)
    # -----
    # Evaluate every candidate literal
    # -----
    for literal in candidates:
        if literal in body:
            continue
        gain, new_positive, new_negative = \
            calculate_gain(
                body,
                literal,
                current_positive,
                current_negative
            )
        print(
            f"{literal:20s} "
            f"Gain = {gain:.4f}"
        )
        # Select literal with highest gain
        if gain > best_gain:

```

```
        best_gain = gain
        best_literal = literal

    # No useful literal found
    if best_literal is None:
        break

    # Add best literal to rule
    body.append(best_literal)

    # Update covered examples
    current_positive = set(
        covered(
            body,
            current_positive
        )
    )

    current_negative = set(
        covered(
            body,
            current_negative
        )
    )

    print(
        "\nSelected literal:",
        best_literal
    )

    print(
        "Positive covered:",
        current_positive
    )
```

```

        print(
            "Negative covered:",
            current_negative
        )

    # Save learned rule
    learned_rules.append(body)

    # Remove positive examples covered by rule
    uncovered_positive -= current_positive

    return learned_rules

# -----
# 9. Learn rules
# -----

rules = foil(
    positive,
    negative
)

# -----
# 10. Display final FOIL rules
# -----

print("\n\n=====")
print("FINAL FOIL RULES")
print("=====")

for i, rule in enumerate(rules, 1):
    print(
        f"Rule {i}: "
        f"can_fly(X) :- "
        f"{' '.join(rule)}."
    )

```

Output:

```

Dataset:
      name  type  has_wings  can_fly
0  sparrow  bird          1         1
1   eagle  bird          1         1
2  penguin  bird          0         0
3     bat  mammal         1         1
4     cat  mammal         0         0
5 butterfly  insect         1         1

Positive Examples:
['sparrow', 'eagle', 'bat', 'butterfly']

Negative Examples:
['penguin', 'cat']

-----
Learning a new rule
-----

Current rule body: []
has_wings(X)      Gain = 2.3399
type(X,bird)      Gain = 0.0000
type(X,mammal)    Gain = -0.4150
type(X,insect)    Gain = 0.5850

Selected literal: has_wings(X)
Positive covered: {'bat', 'sparrow', 'butterfly', 'eagle'}
Negative covered: set()

=====
FINAL FOIL RULES
=====
Rule 1: can_fly(X) :- has_wings(X).

```

4) Read an unsupervised dataset and group the dataset based on similarity based on k-means clustering.

Program:

```

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris

# Step 1: Load the Iris dataset (ignore labels for unsupervised learning)
iris = load_iris()
data = pd.DataFrame(iris.data, columns=iris.feature_names)

```

Step 2: Apply K-Means clustering (3 clusters for 3 species)

```
kmeans = KMeans(n_clusters=3, random_state=42)
```

```
data['Cluster'] = kmeans.fit_predict(data)
```

Step 3: Print cluster centers

```
print("Cluster Centers:")
```

```
print(kmeans.cluster_centers_)
```

Step 4: Plot (using petal length vs petal width for clarity)

```
plt.scatter(data['petal length (cm)'], data['petal width (cm)'],
```

```
           c=data['Cluster'], cmap='viridis', s=50)
```

```
plt.scatter(kmeans.cluster_centers_[0, 2], kmeans.cluster_centers_[0, 3],
```

```
           c='red', marker='X', s=200, label='Centroids')
```

```
plt.xlabel("Petal length (cm)")
```

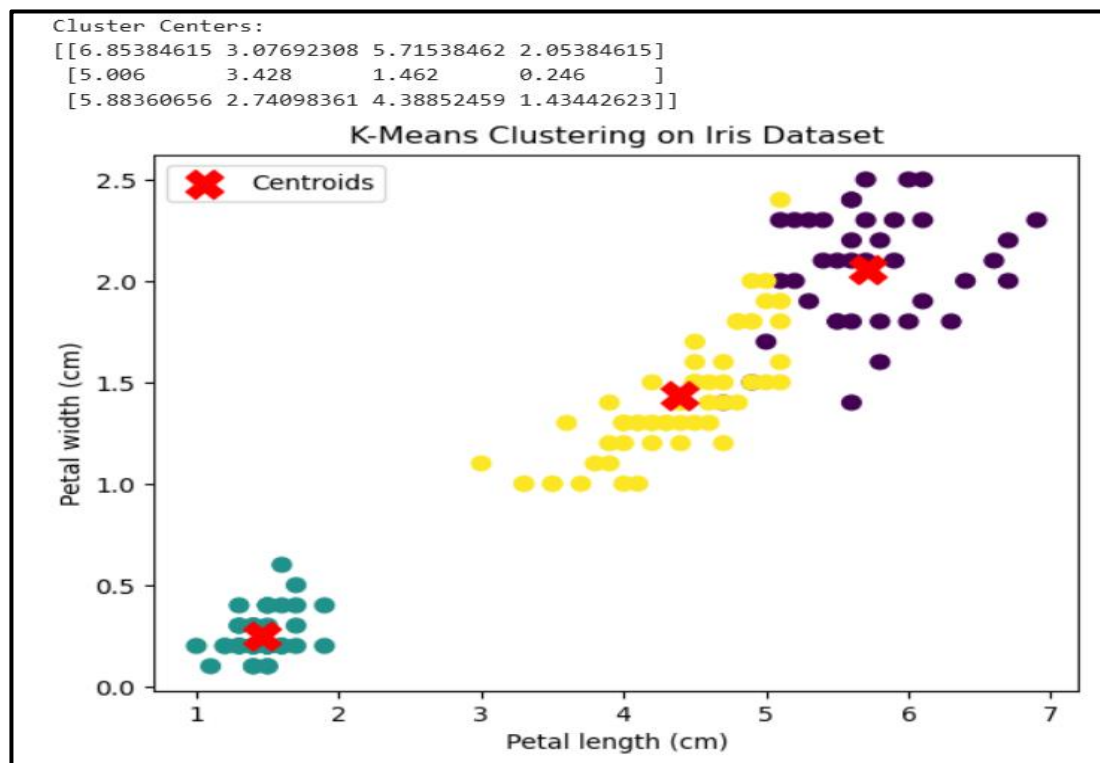
```
plt.ylabel("Petal width (cm)")
```

```
plt.title("K-Means Clustering on Iris Dataset")
```

```
plt.legend()
```

```
plt.show()
```

Output:



5) Read a dataset and perform unsupervised learning using SOM algorithm.

Program:

```
# Step 1: Import libraries
import numpy as np
from minisom import MiniSom
from sklearn import datasets
from sklearn.preprocessing import MinMaxScaler
import matplotlib.pyplot as plt

# Step 2: Load dataset (Iris)
iris = datasets.load_iris()
data = iris.data
target = iris.target

# Step 3: Normalize data (important for SOM)
scaler = MinMaxScaler()
data = scaler.fit_transform(data)

# Step 4: Initialize SOM (10x10 grid, input_len = number of features)
som = MiniSom(x=10, y=10, input_len=4, sigma=1.0, learning_rate=0.5)
som.random_weights_init(data)
som.train_random(data, num_iteration=100) # train for 100 iterations

# Step 5: Visualize results
plt.figure(figsize=(7, 7))
for i, x in enumerate(data):
    w = som.winner(x) # get the winning neuron
    plt.text(w[0] + 0.5, w[1] + 0.5, str(target[i]),
            color=plt.cm.Set1(target[i] / 3),
            fontdict={'weight': 'bold', 'size': 11})
plt.title('SOM clustering of Iris data')
plt.xlim([0, 10])
```

`plt.ylim([0, 10])`

`plt.grid()`

`plt.show()`

Note:

Install the Minisom:

```
[2]: !pip install minisom

Collecting minisom
  Downloading minisom-2.3.5.tar.gz (12 kB)
  Preparing metadata (setup.py): started
  Preparing metadata (setup.py): finished with status 'done'
Building wheels for collected packages: minisom
  Building wheel for minisom (setup.py): started
  Building wheel for minisom (setup.py): finished with status 'done'
  Created wheel for minisom: filename=MiniSom-2.3.5-py3-none-any.whl size=12044 sha256=1d9f66da1f474873301c7d71c339fe8bd5a5d70edd323e9a6e7c89b9c218e8c2
  Stored in directory: c:\users\hp\appdata\local\pip\cache\wheels\0f\8c\45\b7aa56fa6ef11d536d45da775bcc5a2a1c163ff0f8f11990b
Successfully built minisom
Installing collected packages: minisom
Successfully installed minisom-2.3.5
```

Output:



6) Write a function to generate uniform random numbers in the interval $[0, 1]$. Use this function to generate 10 random samples and evaluate $f(x)$ for each sample.

What are the sampled function values? Using the samples generated in the previous step, estimate the integral I using the Monte Carlo method.

Program:

Consider $\rightarrow f(x) = x^2$

```
import random
```

```
import math
```

```
# Function to generate n uniform random numbers in [0, 1]
```

```
def rand_uniform(n):
```

```
    return [random.random() for _ in range(n)]
```

```
# Function to evaluate  $f(x) = x^2$ 
```

```
def f(x):
```

```
    return x**2
```

```
# Set random seed (optional, for reproducibility)
```

```
random.seed(0)
```

```
# Number of samples
```

```
n = 10
```

```
# Generate samples
```

```
samples = rand_uniform(n)
```

```
# Evaluate  $f(x)$  for each sample
```

```
f_values = [f(x) for x in samples]
```

```
# Monte Carlo estimate of the integral  $I = \int_0^1 x^2 dx$ 
```

```
I_estimate = sum(f_values) / n
```

```
# Exact value of the integral
```

```
I_exact = 1 / 3 # since  $\int_0^1 x^2 dx = 1/3$ 
```

```
# Display results
print("Sample\t x\t\t f(x)=x^2")
for i in range(n):
    print(f"{i+1}\t {samples[i]:.6f}\t {f_values[i]:.6f}")

print("\nMonte Carlo estimate of  $I = \int_0^1 x^2 dx$ ")
print(f"Estimated value : {I_estimate:.6f}")
print(f"Exact value : {I_exact:.6f}")
print(f"Absolute error : {abs(I_estimate - I_exact):.6f}")
```

Output:

Sample	x	f(x)=x^2
1	0.844422	0.713048
2	0.757954	0.574495
3	0.420572	0.176880
4	0.258917	0.067038
5	0.511275	0.261402
6	0.404934	0.163972
7	0.783799	0.614340
8	0.303313	0.091999
9	0.476597	0.227145
10	0.583382	0.340335

Monte Carlo estimate of $I = \int_0^1 x^2 dx$		
Estimated value : 0.323065		
Exact value : 0.333333		
Absolute error : 0.010268		

Consider $\rightarrow f(x) = \sin x$

Program:

```
import random, math
def rand_uniform(n):
    return [random.random() for _ in range(n)]

def f(x): # new function
```

```

    return math.sin(x)

random.seed(0)
n = 10
samples = rand_uniform(n)
f_values = [f(x) for x in samples]

I_estimate = sum(f_values) / n
I_exact = 1 - math.cos(1)

print("Sample\t x\t t f(x)=sin(x)")
for i in range(n):
    print(f"{i+1}\t {samples[i]:.6f}\t {f_values[i]:.6f}")

print("\nMonte Carlo estimate of  $I = \int_0^1 \sin(x) dx$ ")
print(f"Estimated value : {I_estimate:.6f}")
print(f"Exact value : {I_exact:.6f}")
print(f"Absolute error : {abs(I_estimate - I_exact):.6f}")

```

Output:

Sample	x	f(x)=sin(x)
1	0.844422	0.747587
2	0.757954	0.687437
3	0.420572	0.408282
4	0.258917	0.256034
5	0.511275	0.489289
6	0.404934	0.393958
7	0.783799	0.705975
8	0.303313	0.298683
9	0.476597	0.458758
10	0.583382	0.550850
Monte Carlo estimate of $I = \int_0^1 \sin(x) dx$		
Estimated value : 0.499685		
Exact value : 0.459698		
Absolute error : 0.039988		

7) Read a dataset and indicate the likelihood of an event occurring using Bayesian Networks.

Program:

```
import pandas as pd
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.estimators import MaximumLikelihoodEstimator
from pgmpy.inference import VariableElimination

# -----
# Step 1: Load a dataset
# -----
data = pd.DataFrame([
    ['Sunny', 'Hot', 'High', 'Weak', 'No'],
    ['Sunny', 'Hot', 'High', 'Strong', 'No'],
    ['Overcast', 'Hot', 'High', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'High', 'Weak', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'Strong', 'No'],
    ['Overcast', 'Cool', 'Normal', 'Strong', 'Yes'],
    ['Sunny', 'Mild', 'High', 'Weak', 'No'],
    ['Sunny', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'Normal', 'Weak', 'Yes'],
    ['Sunny', 'Mild', 'Normal', 'Strong', 'Yes'],
    ['Overcast', 'Mild', 'High', 'Strong', 'Yes'],
    ['Overcast', 'Hot', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'High', 'Strong', 'No']
], columns=['Outlook', 'Temperature', 'Humidity', 'Wind', 'Play'])

# -----
# Step 2: Define Bayesian Network structure
# -----
model = DiscreteBayesianNetwork([
    ('Outlook', 'Play'),
    ('Temperature', 'Play'),
    ('Humidity', 'Play'),
```

```

    ('Wind', 'Play')
])

# -----
# Step 3: Train model using dataset
# -----

model.fit(data, estimator=MaximumLikelihoodEstimator)

# -----
# Step 4: Make inference
# -----

inference = VariableElimination(model)

# Example Query: Probability of Playing given it's Sunny and Humidity is High
query_result = inference.query(variables=['Play'],
                                evidence={'Outlook': 'Sunny', 'Humidity': 'High'})

print("Probability of Play given Outlook=Sunny and Humidity=High:")
print(query_result)

```

Output:

```

INFO:pgmpy: Datatype (N=numerical, C=Categorical Unordered, O=Categorical Ordered) inferred from data:
{'Outlook': 'C', 'Temperature': 'C', 'Humidity': 'C', 'Wind': 'C', 'Play': 'C'}
Probability of Play given Outlook=Sunny and Humidity=High:
+-----+-----+
| Play   | phi(Play) |
+-----+-----+
| Play(No) | 0.7653 |
+-----+-----+
| Play(Yes) | 0.2347 |
+-----+-----+

```

Installation on pgmpy module:

```
[2]: !pip install pgmpy
```

!pip install --upgrade pgmpy  For upgrading pgmpy

8) Refer to the dataset in question 7 and indicate inferences based on the sequence of steps .

Program:

```
import pandas as pd
from pgmpy.models import DiscreteBayesianNetwork
from pgmpy.estimators import MaximumLikelihoodEstimator
from pgmpy.inference import VariableElimination

# -----
# Step 1: Load dataset
# -----
data = pd.DataFrame([
    ['Sunny', 'Hot', 'High', 'Weak', 'No'],
    ['Sunny', 'Hot', 'High', 'Strong', 'No'],
    ['Overcast', 'Hot', 'High', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'High', 'Weak', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'Strong', 'No'],
    ['Overcast', 'Cool', 'Normal', 'Strong', 'Yes'],
    ['Sunny', 'Mild', 'High', 'Weak', 'No'],
    ['Sunny', 'Cool', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'Normal', 'Weak', 'Yes'],
    ['Sunny', 'Mild', 'Normal', 'Strong', 'Yes'],
    ['Overcast', 'Mild', 'High', 'Strong', 'Yes'],
    ['Overcast', 'Hot', 'Normal', 'Weak', 'Yes'],
    ['Rain', 'Mild', 'High', 'Strong', 'No']
], columns=['Outlook', 'Temperature', 'Humidity', 'Wind', 'Play'])

# -----
# Step 2: Define Bayesian Network structure
# -----
model = DiscreteBayesianNetwork([
    ('Outlook', 'Play'),
    ('Temperature', 'Play'),
    ('Humidity', 'Play'),
    ('Wind', 'Play')])
```

])

Step 3: Train the model using MLE

model.fit(data, estimator=MaximumLikelihoodEstimator)

Step 4: Perform Inference

inference = VariableElimination(model)

Step 5: Make multiple inferences

Example 1: P(Play | Outlook=Sunny, Humidity=High)

q1 = inference.query(variables=['Play'], evidence={'Outlook': 'Sunny', 'Humidity': 'High'})

print("\n1: P(Play | Outlook=Sunny, Humidity=High):")

print(q1)

Example 2: P(Play | Outlook=Rain, Wind=Weak)

q2 = inference.query(variables=['Play'], evidence={'Outlook': 'Rain', 'Wind': 'Weak'})

print("\n2: P(Play | Outlook=Rain, Wind=Weak):")

print(q2)

Example 3: P(Play | Outlook=Overcast)

q3 = inference.query(variables=['Play'], evidence={'Outlook': 'Overcast'})

print("\n3: P(Play | Outlook=Overcast):")

print(q3)

Example 4: P(Play | Temperature=Cool, Humidity=Normal)

```
q4 = inference.query(variables=['Play'], evidence={'Temperature': 'Cool', 'Humidity':
'Normal'})
print("\n4: P(Play | Temperature=Cool, Humidity=Normal):")
print(q4)
```

Example 5: P(Play | Wind=Strong)

```
q5 = inference.query(variables=['Play'], evidence={'Wind': 'Strong'})
print("\n5: P(Play | Wind=Strong):")
print(q5)
```

Output:

```
INFO:pgmpy: Datatype (N=numerical, C=Categorical Unordered, O=Categorical Ordered) inferred from data:
{'Outlook': 'C', 'Temperature': 'C', 'Humidity': 'C', 'Wind': 'C', 'Play': 'C'}
```

```
1: P(Play | Outlook=Sunny, Humidity=High):
```

Play	phi(Play)
Play(No)	0.7653
Play(Yes)	0.2347

```
2: P(Play | Outlook=Rain, Wind=Weak):
```

Play	phi(Play)
Play(No)	0.2143
Play(Yes)	0.7857

```
3: P(Play | Outlook=Overcast):
```

Play	phi(Play)
Play(No)	0.3418
Play(Yes)	0.6582

```
4: P(Play | Temperature=Cool, Humidity=Normal):
```

Play	phi(Play)
Play(No)	0.3112
Play(Yes)	0.6888

```
5: P(Play | Wind=Strong):
```

Play	phi(Play)
Play(No)	0.5000
Play(Yes)	0.5000